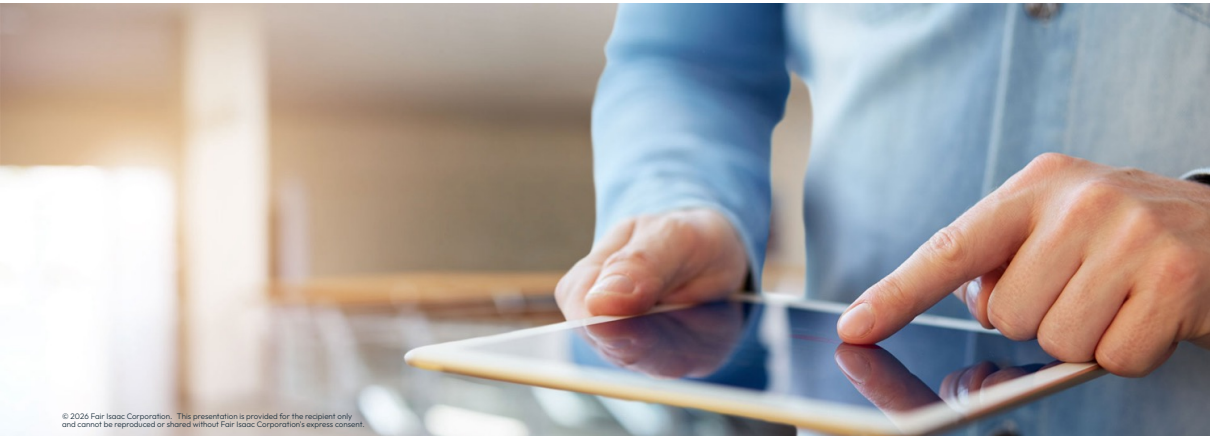
Multiple MIP solutions



Multiple MIP solutions

Multiple MIP solutions

- When solving a MIP problem, FICO® Xpress will typically find a number of MIP solutions before finding the optimal solution:
  - Sometimes you may want to use these intermediate solutions, or give the user the opportunity to stop the search early and use one of them
- Once it has found a MIP solution, it will only search for better MIP solutions:
  - The value of MIP solutions saved in the search always improves in the order that they are found
- Finding multiple MIP solutions is the most common use case of library callbacks

Note: *Xpress is designed to find good MIP solutions. It cannot be used to find all possible MIP solutions to a problem*

Callback functions

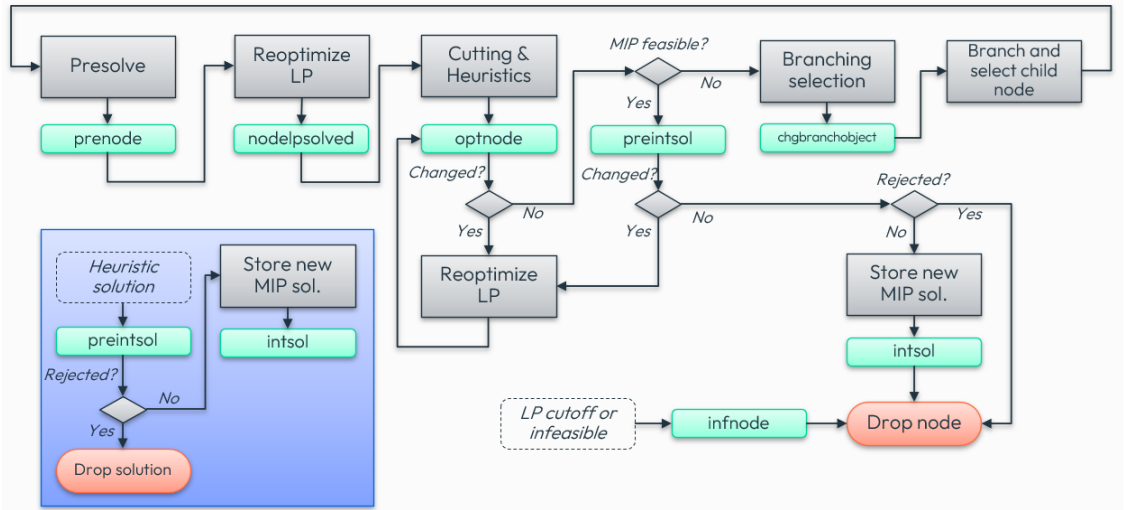
- To capture all MIP solutions found by Xpress, you need to declare a callback function
- The library callbacks are a collection of functions which allow user–defined routines to be specified to Xpress Optimizer:
  - Called at various stages during the optimization process, prompting the Optimizer to return to the user's program before continuing with the solution algorithm
  - In Python, names of functions for defining callbacks are of the form problem.add*Callback()
- With the MIP solution callback method problem.addIntsolCallback(), you can access any new integer solution found, display it, store it, output it, allow the user to examine it and stop the search early, *etc.*

Main MIP-related callbacks

- Logging callbacks: message, lplog, barlog, cutlog, miplog
- Tracking the MIP branch-and-bound tree: newnode, infnode, nodecutoff
- Callbacks that influence the MIP search:
  - prenode, nodelpsolved: tighten bounds, add solutions
  - optnode: tighten bounds, add solutions, add cuts, add branching candidates
  - chgbranchobject: override Xpress selected branching object
  - preintsol: tighten bounds, add solutions, add cuts, add branching candidates, reject MIP solution, adjust cutoff
- Controls can be changed deterministically within node specific callbacks
- Use problem.getCallbackSolution() inside the defined function to retrieve the context specific solution

Note: *Adding cuts or branching candidates is not yet supported for Global solves*

MIP Node Callback Flow



Using callback functions in Python

- Steps for using callbacks using the Python API:
  1. Define a callback function (say `exportsol`) that is to be run at certain points in time (for p.addIntsolCallback(), every time a new integer solution is found):

```
def exportsol(prob, data, soltype, cutoff):
   with open('solutions.txt', 'a') as file:
      file.write(f"New solution: {prob.getCallbackSolution()}")
```

  2. Call the corresponding problem.add*Callback() method with `exportsol` as its argument

```
p.addIntsolCallback(exportsol, data)   # assume data defined elsewhere
```

  3. Run the p.optimize() command that launches the appropriate solver

Using callback functions in Python

- Steps for using callbacks using the Python API:
  1. Define a callback function (say `exportsol`) that is to be run at certain points in time (for p.addIntsolCallback(), every time a new integer solution is found):

```
def exportsol(prob, data, soltype, cutoff):
   with open('solutions.txt', 'a') as file:
      file.write(f"New solution: {prob.getCallbackSolution()}")
```

  2. Call the corresponding problem.add*Callback() method with `exportsol` as its argument

```
p.addIntsolCallback(exportsol, data)  # assume data defined elsewhere
```

  3. Run the p.optimize() command that launches the appropriate solver

- A callback function is passed once as an argument and used possibly many times while a solver is running, and receives:
  - A `problem object` declared with `p = xp.problem()`
  - A user-defined `data object` to read and/or modify information within the callback
    - The data object may be optional, depending on the callback type

MIP solution callback

- Example for a callback function named `intsolcb` that is called every time a new integer solution is found via the p.addIntsolCallback() method:

```
import xpress as xp

def printsolcb(prob, data, soltype, cutoff):
  # callback to be used when an integer solution is found defined here
   print(f"Solution: {prob.getCallbackSolution()}")

p = xp.problem()
p.read('myprob.lp')  # reads in a problem, let's say a MIP

p.addIntsolCallback(printsolcb, data)  # assume 'data' defined elsewhere
p.optimize()
```

Note: *While the* `function` *argument is necessary for all p.add*Callback() functions, the* `data` *object can be specified as* `None`*. In that case, the callback function will be run with* `None` *as its data argument*

Finding multiple MIP solutions

- Xpress Optimizer finds increasingly better solutions:
  - New solutions that are valued the same or worse than the current best are discarded by the Optimizer by default
  - The value of the mipaddcutoff control parameter can be added to the current best solution's objective value to set a new cutoff
  - Since it is an absolute value, the sign must be set depending on the objective sense. Example for a maximization problem:

```
p.controls.mipaddcutoff = -0.5 # accept solutions 0.5 worse than current cutoff
```

- Setting mipaddcutoff to a negative (positive) value for a maximization (minimization) problem allows the solver to find worse solutions

Adding MIP solutions

- Provide Xpress Optimizer with a new feasible, infeasible or partial MIP solution via the problem.addMipSol() method:

```
p.addMipSol(solval, colind, name)
```

  - If a partial solution is provided:
    - if values are given for all discrete columns, these will be fixed to those values and the continuous part optimized
    - if some integer values are not defined, a local search will be run in an attempt to find any remaining integer feasible values
  - If the provided solution is found to be infeasible:
    - local search heuristic will be run in an attempt to find a close feasible integer solution
  - The method can also be called from callbacks

Hint: *Check the online TSP example that demonstrates the basic use of providing a full, feasible solution to warm-start a MIP solve*

Python Notebook [PN-5]: Multiple MIP solutions

- Navigate to Exercise 5 - Multiple MIP solutions in the notebook

1. Storing multiple MIP solutions via callbacks:
   - Examine the callback functions printsol, exportsol, validatesol to understand what they are programmed to do
   - Run the code cell with the optimization model three times calling a different callback function each time and analyze the output(s)
   - In the last code cell of this part, set the mipaddcutoff control value to `-0.5` and re-run the MIP model using `printsol` as callback:
     - How many additional solutions are found by the Optimizer ?
2. Adding MIP solutions:
   - Run the code cell in part 5.2 and analyze the output log:
     - Has any of the randomly generated solutions resulted feasible ?
     - If not, has a feasible solution been obtained after applying local search to any of those initially infeasible solutions ?

Thank You

www.fico.com/optimization